
Should Your Draft Model Be Ternary?

Eren Menges
New York University

Abstract

Speculative decoding speeds up LLM inference, and on memory-bound devices, the speedup depends on the draft model’s memory footprint. Ternary models occupy far less memory, but their use as draft models remains untested because comparing FP16 and ternary QAT models requires training both models identically apart from precision. We measure and compare the acceptance rate of 21 draft models spanning FP16, ternary-via-QAT, and ternary-via-PTQ against two 3.9B targets across three domains, using models from SpectraSuite. We validate the analysis against end-to-end speculative decoding with draft length $k = 5$. We find that an FP16 draft needs $1.74\times$ as much memory to match the best ternary draft’s acceptance rate (FP16 target, web domain). Additionally, we find that the ternary-draft penalty at matched parameters depends on the target’s precision: at the largest drafts (1.5B, web domain), it nearly vanishes against a ternary target (-0.0031) while remaining substantial against an FP16 target (-0.0348). Despite being identical to its QAT counterpart in architecture, parameter count, and memory footprint (1.5B, web domain), the PTQ model attains an acceptance rate of 0.0823 compared to the QAT model’s 0.7327.

1 Introduction

LLM inference is costly, particularly on consumer devices. Speculative decoding enables faster inference with no output quality loss [Leviathan et al., 2023, Chen et al., 2023]. It uses a smaller, cheaper draft model to sequentially generate k predictions. The target model then verifies all k predictions in a single forward pass instead of k passes. In the best scenario, every prediction is accepted, and that one pass yields $k + 1$ tokens instead of one.

Especially on consumer devices, which are typically memory-bandwidth-bound, the speedup is limited by the memory footprint of draft models. Quantization reduces a model’s memory footprint at a small cost in quality. We evaluate one such technique, ternary Quantization-Aware Training (QAT), to determine whether it speeds up speculative decoding.

To our knowledge, no controlled comparison between FP16 and ternary QAT draft models exists. Comparing ternary models and higher-precision models such as FP16 models requires the models to have identical architecture, tokenizer, and training sets, which entails substantial pretraining from scratch. The controlled comparison was made possible by SpectraSuite [Kaushal et al., 2024]. The SpectraSuite family includes matched FP16 and ternary-QAT variants, identical in every way except training pipeline and precision. We evaluate SpectraSuite’s models both analytically and in end-to-end speculative decoding runs.

We find that ternary drafts typically match or exceed FP16 acceptance rates at equal or smaller footprints. Moreover, we find that against a ternary target, the penalty of using a ternary draft nearly vanishes, while it stays substantial against an FP16 target. Finally, we find that naive ternary Post-Training Quantization (PTQ) makes speculative decoding slower than autoregressive decoding.

2 Experimental setup

We evaluate the models of the SpectraSuite family [Kaushal et al., 2024]. We use models from two sub-families: FloatLM, which is trained with FP16 precision, and TriLM, which is trained with QAT at ternary precision. All models were trained on the same 300B tokens, have the same LLaMA-style architecture and share the same tokenizer. Draft models span seven sizes from 99M to 1.5B parameters; target models have 3.9B parameters. We evaluate 14 pretrained draft models (seven FP16, seven ternary) plus seven PTQ variants against two target models. We construct PTQ models by applying per-matrix absmean round-to-nearest (RTN) to SpectraSuite’s FP16 models.

We evaluate speculative decoding acceptance rate across three domains and 1,024 prompts: the web domain (512 prompts), the code domain (256), and the chat domain (256). We define α_t , the acceptance probability at token position t , as $\alpha_t = \sum_{x \in V} \min(p_t(x), q_t(x))$ where V is the models’ vocabulary, x is a token in V , $p_t(x)$ is the probability assigned to token x by the target model at position t , and $q_t(x)$ is the probability assigned to token x by the draft model at position t .

We define analytical α as α_{prompt} averaged over all prompts that a specific combination was evaluated on. α is used interchangeably with acceptance rate, since α_t equals the probability that a token sampled from q_t is accepted by the speculative sampling rejection rule. This identity was derived in the original speculative decoding paper [Leviathan et al., 2023].

We model the expected tokens per round as $\mathbb{E}[\text{tokens/round}] = \sum_{i=0}^k \alpha^i = \frac{1-\alpha^{k+1}}{1-\alpha}$, where a round is one cycle of draft proposing and target verifying, and k is the number of tokens drafted per round. We assume that each drafted token is accepted independently with probability α [Leviathan et al., 2023].

Speedup is computed relative to the standard autoregressive, target-only decoding ($1\times$). We express all costs in units of one target forward pass. We assume the cost is memory-bound, so cost of one forward pass of a model $\approx$ cost of streaming the model’s weights. Therefore, we derive cost c as $\frac{\text{memory footprint of the draft model}}{\text{memory footprint of the target model}}$. Scoring k tokens (and obtaining the bonus/resample distribution) costs 1, since it only takes one target forward pass. Using the above definitions, we calculate speedup using the formula $\frac{\mathbb{E}[\text{tokens/round}]}{1+k \cdot c}$ [Leviathan et al., 2023, Chen et al., 2023]. We use $k = 5$ throughout this paper. Finally, we validate the analytical part of this experiment with end-to-end speculative decoding runs against the FloatLM_3.9B target. (Setup details at Appendix A or GitHub.)

3 Results

Unless noted otherwise, reported numbers are for the web domain against the FloatLM_3.9B target; chat and code show the same pattern (Appendix Tables 2–6).

3.1 Ternary drafts match or outperform FP16 drafts at matched memory

At matched parameter count, ternary drafts attain lower α at every size tested. On memory-bound hardware, however, a draft’s cost is set by its memory footprint, not its parameter count. Evaluated at matched memory footprint, ternary drafts achieve higher α at every point on the 200–740 MB overlap, except the 200 MB boundary (tie) (Figure 1). The advantage widens with scale: an FP16 draft requires $1.31\times$ the memory to match a 278 MB ternary draft, rising to $1.74\times$ at 740 MB (1.5B parameters) (Appendix Table 3).

For example, TriLM_1.5B (740 MB) occupies less memory than FloatLM_390M (785 MB) yet attains a higher α (0.7327 vs. 0.7098) (Appendix Tables 2 and 6).

3.2 The ternary penalty at matched parameters depends on target precision

Ternary drafts incur an α penalty relative to their FP16 counterparts at matched parameters. We calculate it as $\alpha(\text{TriLM draft}) - \alpha(\text{FloatLM draft})$.

However, this penalty depends on the precision of the target model. For example, at 99M, the penalties against the FP16 and ternary targets are -0.0540 and -0.0515 , respectively. As the parameter count increases, the penalty against the ternary target rapidly narrows toward zero whereas the penalty

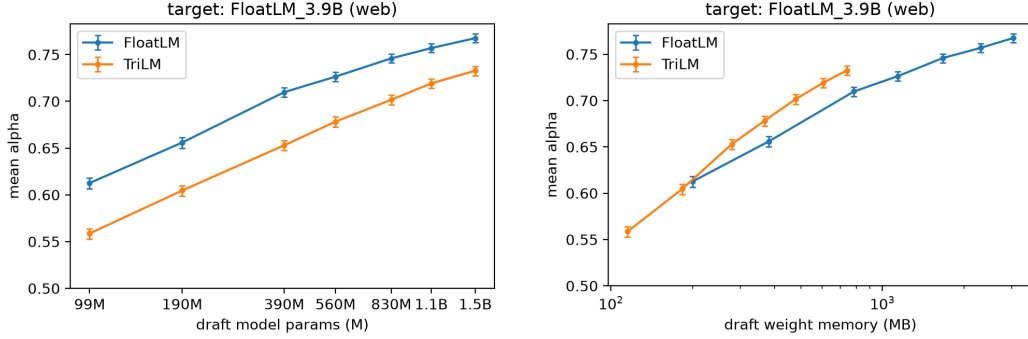


Figure 1: Mean acceptance rate α against draft parameter count (left) and draft memory footprint (right), web domain, FloatLM_3.9B target.

against the FP16 target narrows much more slowly (Appendix Figure 3). At 1.5B, the penalty against the ternary target is -0.0031 , whereas the penalty against the FP16 target is -0.0348 . The penalty -0.0031 is at the scale of the noise floor, but the paired gap comparison stays meaningful since the target-side α noise is shared by both drafts within each paired difference and the CI still excludes zero.

At 1.5B, switching the target from FP16 to ternary makes the FP16 draft worse (-0.0123) and the ternary draft better ($+0.0194$; Appendix Table 1). This suggests two components of the penalty: reduced draft capacity and precision mismatch between the target and the draft models (Appendix Tables 2, 4, and 5).

3.3 PTQ collapses the draft

At fixed memory footprint, the quantization method matters. Ternary drafts trained with QAT retain high α , while the ternary drafts quantized via PTQ collapse. Appendix Figure 4 shows that PTQ severely degrades analytical α . At a memory footprint of ~ 740 MB, TriLM_1.5B and PTQ_1.5B have identical architecture, parameter counts, and memory footprint. The PTQ model attains $\alpha = 0.0823$ compared to the QAT ternary model’s $\alpha = 0.7327$ (Appendix Table 2).

3.4 Ternary drafts sustain speedup across the memory range

The speedup of ternary drafts remains within $2.05\text{--}2.28\times$ across the full 115–740 MB range, while FP16 peaks at $2.17\times$ with a 99M draft and falls to $1.18\times$ by 1.5B (Figure 2). The speedup of FP16 drafts depends strongly on model parameter count, while the speedup of ternary drafts does not.

The speedups of all draft models against the ternary target are lower than the speedups against the FP16 target. Because the ternary target model’s footprint is smaller, the relative draft cost c is higher.

PTQ slows generation. PTQ_1.5B at 740 MB generates 1.09 tokens per round, which amounts to a $0.74\times$ speedup, slower than standard autoregressive decoding (Appendix Table 6).

3.5 Validation

Predicted and measured tokens per round track each other closely across 27 distinct combinations of domains and draft models. Mean deviation is -0.0394 on values from 1.01 to 3.97, with a maximum absolute deviation of 0.19. Measured values fall below predicted in 21 of the 27 combinations (Appendix Table 7 and Figure 5). One likely cause is that acceptance declines across draft depth, whereas the analytical estimate sees target-generated context at every position.

4 Related work

Speculative decoding has been proposed to speed up inference by using a smaller draft model [Leviathan et al., 2023, Chen et al., 2023]. BitNet and BitNet b1.58 explored transformers with 1-bit

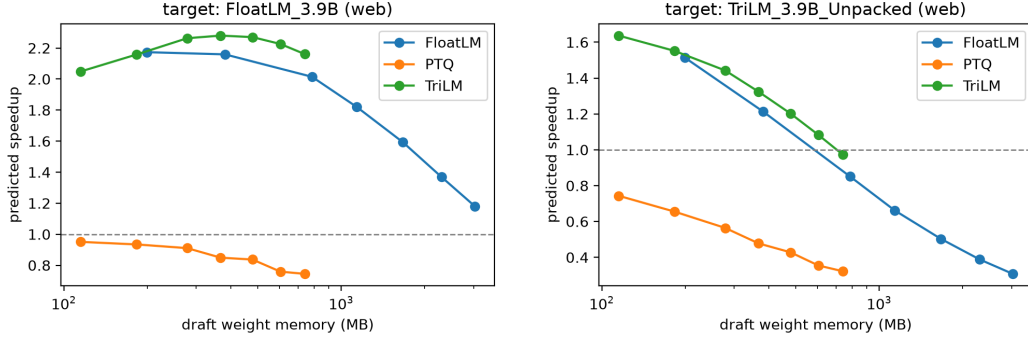


Figure 2: Predicted speedup against draft memory footprint, web domain, PTQ included, against a FloatLM_3.9B target (left) and TriLM_3.9B target (right).

and ternary weights [Wang et al., 2023, Ma et al., 2024]. Spectra explores training ternary models and comparing them to the identical-except-precision FP16 counterparts on a much larger scale; Spectra 1.1 proposes a 2-bit packing scheme for storing weights [Kaushal et al., 2024, Vaidhya et al., 2025].

Mahmoud [2026] measures acceptance rates across multiple domains. Applying PTQ on parts of speculative decoding has been explored by QSpec, QuantSpec, and ML-SpecQD [Zhao et al., 2025, Tiwari et al., 2025, Georganas et al., 2025]. Moreover, ML-SpecQD authors propose exploring 2-bit drafts as future work [Georganas et al., 2025]. TriSpec targets verification cost, routing between a smaller full-precision proxy and the target; its “ternary” means three models, not weight precision [Jiang et al., 2026].

These works mainly focus on quantization after training. Whether a model trained at low precision with QAT makes a better draft has not been tested.

5 Limitations

PTQ. The PTQ method used is naive absmean RTN, which is the same quantization function TriLM uses during QAT. This makes it the right control for isolating training-time vs. post-training quantization under the same weight format. However, this method is fairly primitive. Stronger methods with calibration or error compensation (e.g., GPTQ, AWQ) would likely collapse far less. Our results therefore show that naive RTN ternarization fails at these scales, and that QAT avoids this failure. They do not show that there is no post-training path to a ternary draft. **Scope.** Although two different targets were used, only one target size was used (3.9B), temperature was fixed at 1.0, k was fixed at $k = 5$, EOS was masked to prevent early termination, only three domains were tested, tasks were continuation-only, and a fixed prefix starting at token index 500 was used for every document. **Lack of wall-clock speedup measurements.** The models of SpectraSuite are released in unpacked format. This means that the weights are ternary, but are stored in FP16 format on disk. Therefore, speedups are predicted from the cost model rather than measured in wall-clock time. The cost is only computed from memory footprint, assuming that speedup is memory-bound.

6 Conclusion and future work

We find that ternary drafts typically surpass FP16 drafts at matched memory footprint; that the ternary penalty at matched parameters nearly vanishes against a ternary target but persists against an FP16 target; and that naive PTQ collapses the draft whereas QAT does not. On memory-bound hardware, our results support choosing ternary QAT draft models over FP16 draft models for higher acceptance rates and speedup.

Future work includes real-time wall-clock speedup measurements, comparing ternary drafts with INT4/INT8 PTQ drafts that are more common in deployment, and testing drafts with larger ternary targets.

References

- Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling. arXiv:2302.01318, 2023. URL <https://arxiv.org/abs/2302.01318>.
- Evangelos Georganas, Dhiraj Kalamkar, Alexander Kozlov, and Alexander Heinecke. ML-SpecQD: Multi-level speculative decoding with quantized drafts. arXiv:2503.13565, 2025. URL <https://arxiv.org/abs/2503.13565>.
- Haoyun Jiang, Junqi He, Feng Hong, Xinlong Yang, Jianwei Zhang, Zheng Li, Zhengyang Zhuge, Zhiyong Chen, Bo Han, Junyang Lin, and Jiangchao Yao. Trispec: Ternary speculative decoding via lightweight proxy verification. arXiv:2601.23180, 2026. URL <https://arxiv.org/abs/2601.23180>.
- Ayush Kaushal, Tejas Vaidhya, Arnab Kumar Mondal, Tejas Pandey, Aaryan Bhagat, and Irina Rish. Spectra: Surprising effectiveness of pretraining ternary language models at scale. arXiv:2407.12327, 2024. URL <https://arxiv.org/abs/2407.12327>.
- Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett, editors, *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 19274–19286. PMLR, 23–29 Jul 2023. URL <https://proceedings.mlr.press/v202/leviathan23a.html>.
- Shuming Ma, Hongyu Wang, Lingxiao Ma, Lei Wang, Wenhui Wang, Shaohan Huang, Li Dong, Ruiping Wang, Jilong Xue, and Furu Wei. The era of 1-bit LLMs: All large language models are in 1.58 bits. arXiv:2402.17764, 2024. URL <https://arxiv.org/abs/2402.17764>.
- Saif Mahmoud. Acceptance dynamics across cognitive domains in speculative decoding. arXiv:2604.14682, 2026. URL <https://arxiv.org/abs/2604.14682>.
- Rishabh Tiwari, Haocheng Xi, Aditya Tomar, Coleman Hooper, Sehoon Kim, Maxwell Horton, Mahyar Najibi, Michael W. Mahoney, Kurt Keutzer, and Amir Gholami. QuantSpec: Self-speculative decoding with hierarchical quantized kv cache. arXiv:2502.10424, 2025. URL <https://arxiv.org/abs/2502.10424>.
- Tejas Vaidhya, Ayush Kaushal, Vineet Jain, Francis Couture-Harpin, Prashant Shishodia, Majid Behbahani, Irina Rish, and Yuriy Nevmyvaka. Scaling laws and efficient inference for ternary language models. In *Sparsity in LLMs (SLLM): Deep Dive into Mixture of Experts, Quantization, Hardware, and Inference*, 2025. URL <https://openreview.net/forum?id=hz8ryFZV0u>.
- Hongyu Wang, Shuming Ma, Li Dong, Shaohan Huang, Huaijie Wang, Lingxiao Ma, Fan Yang, Ruiping Wang, Yi Wu, and Furu Wei. BitNet: Scaling 1-bit transformers for large language models. arXiv:2310.11453, 2023. URL <https://arxiv.org/abs/2310.11453>.
- Juntao Zhao, Wenhao Lu, Sheng Wang, Lingpeng Kong, and Chuan Wu. QSpec: Speculative decoding with complementary quantization schemes, 2025. URL <https://openreview.net/forum?id=RCiwz7WqUU>.

A Additional setup details

Naive absmean RTN is the technique used by SpectraSuite while training the TriLM family with QAT; therefore, the PTQ application serves as a control. We apply naive absmean RTN only to seven block projections: the query/key/value/output attention projections, and gate/up/down MLP projections. Embeddings, norms, and the output LM head are left unquantized.

The web dataset comes from allenai/c4, which itself is derived from Common Crawl. The code dataset is a subset of HuggingFaceCode/stack-v3-train, which comes from public GitHub repositories. Only code files with permissive licenses were used. The chat dataset comes from allenai/WildChat-4.8M, which consists of conversations between users and ChatGPT. We use the English C4 validation split for web; for code we keep only files timestamped after January 2025, which postdates the models’ training data and guards against contamination; and for chat we keep English-only conversations rendered as alternating User:/Assistant: turns.

Documents are sampled from each dataset using a fixed seed. We start the prompt prefixes at token index 500 of every chosen dataset document. The prompts are 256 tokens long, and the target models generate tokens 256–511. During generation and scoring, we mask the EOS token at every position by setting its pre-softmax logit to $-\infty$ so that the generation continues until position 511. We use temperature 1.0. In total, 21 draft models, two target models, and three domains provide 126 target-draft-domain combinations.

At position t , p_t and q_t are conditioned on the 256-token prefix plus the target-generated continuation up to position $t - 1$. This makes the computation of α teacher-forced. We compute the mean acceptance probability per prompt α_{prompt} by averaging α_t over the 256 positions that the target generated.

In Section 3.5, for validation, we evaluate the expected tokens per round formula per prompt at α_{prompt} and average the per-prompt predictions. In Section 3.4, for speedup, we evaluate the expected tokens per round formula per combination at the analytical α of that combination. By convexity, the latter understates the average of per-prompt predictions, making the speedup estimates conservative.

We estimate 95% confidence intervals (CIs) of analytical α with a nonparametric bootstrap via resampling of α_{prompt} with replacement (percentile method, 2,000 resamples). Additionally, we pair up the FP16 and ternary draft models by their parameter counts and evaluate the penalty of using a ternary draft model, the analytical α difference $\alpha(\text{TriLM draft}) - \alpha(\text{FloatLM draft})$, for each pair. We use the same CI estimation but with per-prompt differences. For α , we measure the noise floor by running the 3.9B targets as drafts against themselves. Ideally this should result in $\alpha = 1$. Self- α is 0.982–0.989 for FloatLM and 0.993–0.995 for TriLM; the noise floor is thus 0.006–0.018 depending on family and domain. The noise has two sources: the same BF16 weights traversed by two numerically distinct paths (incremental KV-cached generation vs. a single full-sequence scoring pass) during generation and scoring, and the FP16 rounding of the stored target logits.

For TriLM and PTQ models, only the seven block projections use ternary weights and we assume they occupy 2 bits per weight [Vaidhya et al., 2025]. We assume the other weights of these models and all of the weights of FloatLM models occupy 16 bits per weight.

For validation, we randomly select a total of 256 prompts from our datasets (128 web prompts, 64 code prompts, 64 chat prompts). We run real-time speculative decoding using nine draft models $\{99\text{M}, 560\text{M}, 1.5\text{B}\} \times \{\text{FloatLM}, \text{TriLM}, \text{PTQ}\}$ on each prompt, where we measure accepted draft tokens (plus the bonus/resample token) per round.

B Additional results

Table 1: Acceptance rate of the 1.5B drafts against both targets, web domain. Matching draft and target precision helps the ternary draft.

draft (1.5B)	FloatLM_3.9B target	TriLM_3.9B target	Δ
FloatLM_1.5B	0.7675	0.7552	-0.0123
TriLM_1.5B	0.7327	0.7521	+0.0194
Δ	-0.0348	-0.0031	

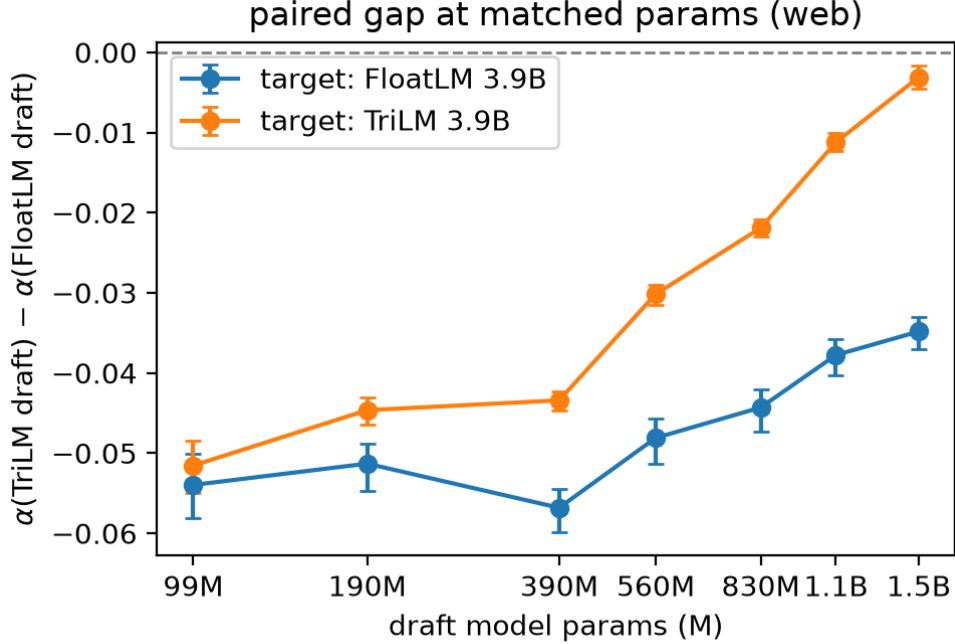


Figure 3: The difference between mean acceptance rates of ternary and FP16 draft (the analytical α gap) against draft parameter count, web domain, against both FloatLM_3.9B and TriLM_3.9B targets.

Table 2: Mean acceptance rate by draft size, family, and domain, for both targets. Temperature 1.0. EOS masked via setting pre-softmax logit to $-\infty$. The entries at size 3900, where the same 3.9B model is used as both a target and a draft, measure the pipeline’s noise floor.

target	params (M)	FloatLM			TriLM			PTQ		
		web	chat	code	web	chat	code	web	chat	code
FloatLM_3.9B	99	0.6126	0.6644	0.7190	0.5586	0.5985	0.6562	0.0188	0.0106	0.0048
	190	0.6559	0.7023	0.7515	0.6046	0.6501	0.7018	0.0396	0.0291	0.0153
	390	0.7098	0.7474	0.7867	0.6529	0.6979	0.7430	0.0647	0.0391	0.0223
	560	0.7263	0.7625	0.8006	0.6782	0.7209	0.7620	0.0424	0.0276	0.0151
	830	0.7459	0.7810	0.8137	0.7016	0.7421	0.7799	0.0809	0.0578	0.0311
	1100	0.7569	0.7925	0.8224	0.7191	0.7573	0.7933	0.0437	0.0306	0.0203
	1500	0.7675	0.7998	0.8283	0.7327	0.7704	0.8029	0.0823	0.0530	0.0361
	3900	0.9819	0.9854	0.9887	0.7643	0.7991	0.8264	-	-	-
TriLM_3.9B	99	0.6219	0.6650	0.7233	0.5703	0.6046	0.6650	0.0198	0.0115	0.0051
	190	0.6624	0.7029	0.7535	0.6178	0.6558	0.7106	0.0411	0.0306	0.0164
	390	0.7113	0.7422	0.7866	0.6679	0.7040	0.7530	0.0678	0.0419	0.0241
	560	0.7238	0.7551	0.7993	0.6936	0.7285	0.7739	0.0442	0.0301	0.0154
	830	0.7410	0.7701	0.8108	0.7191	0.7501	0.7927	0.0842	0.0600	0.0339
	1100	0.7493	0.7788	0.8175	0.7381	0.7661	0.8060	0.0454	0.0337	0.0200
	1500	0.7552	0.7838	0.8204	0.7521	0.7809	0.8179	0.0860	0.0560	0.0354
	3900	0.7634	0.7944	0.8293	0.9927	0.9936	0.9945	-	-	-

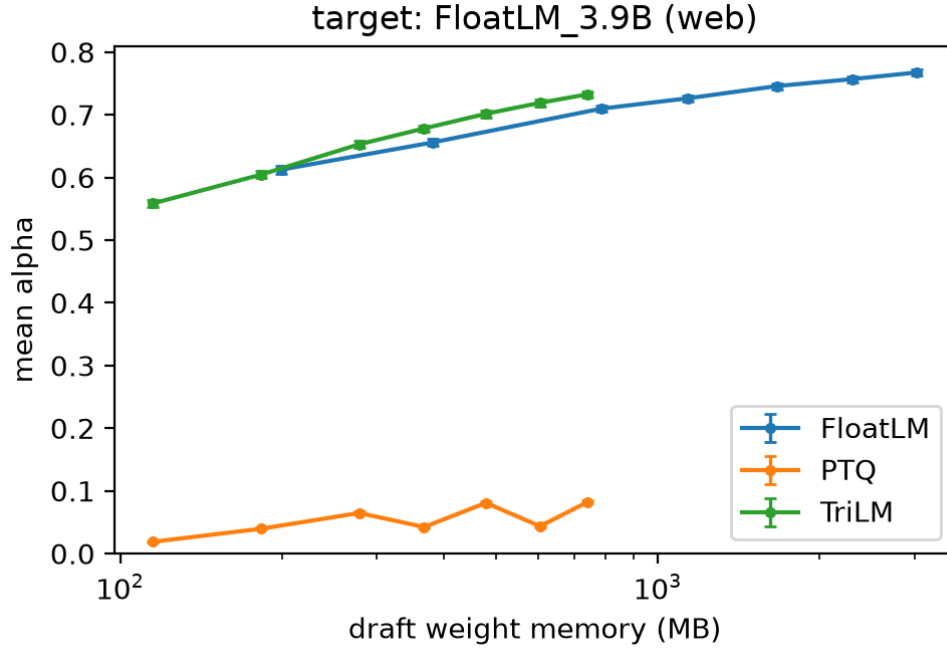


Figure 4: Mean acceptance rate α against draft memory footprint, web domain, PTQ included, FloatLM_3.9B target.

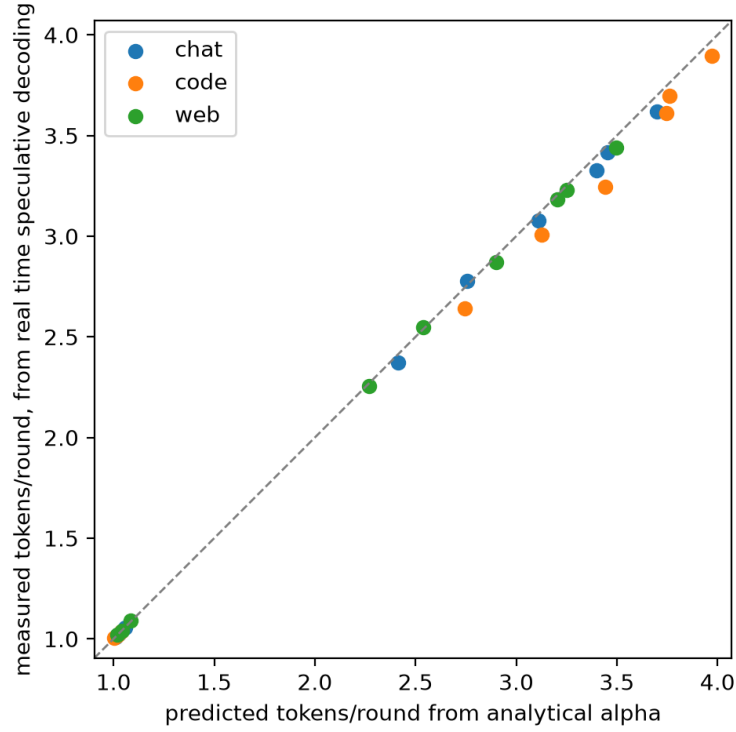


Figure 5: Measured tokens per round from real-time speculative decoding against predicted tokens per round computed from the analytical α , all domains, PTQ included, FloatLM_3.9B target.

Table 3: Memory an FP16 draft needs to match each ternary draft’s acceptance rate, via interpolation of the FP16 α -vs-memory curve. The two smallest ternary drafts fall below the FP16 grid and are omitted. Both targets, all domains.

target	domain	ternary (MB)	FP16 equiv. (MB)	ratio
FloatLM_3.9B	web	278	365	1.31
	web	368	514	1.40
	web	479	704	1.47
	web	604	968	1.60
	web	740	1289	1.74
	chat	278	354	1.27
	chat	368	514	1.40
	chat	479	721	1.51
	chat	604	1001	1.66
	chat	740	1340	1.81
	code	278	322	1.16
	code	368	473	1.29
	code	479	682	1.42
	code	604	936	1.55
	code	740	1219	1.65
TriLM_3.9B	web	278	413	1.48
	web	368	604	1.64
	web	479	990	2.07
	web	604	1564	2.59
	web	740	2621	3.54
	chat	278	389	1.40
	chat	368	611	1.66
	chat	479	986	2.06
	chat	604	1505	2.49
	chat	740	2593	3.50
	code	278	377	1.35
	code	368	595	1.62
	code	479	940	1.96
	code	604	1420	2.35
	code	740	2397	3.24

Table 4: α penalty of a ternary draft against the FP16 target at matched parameter count, $\alpha(\text{TriLM}) - \alpha(\text{FloatLM})$, for all domains. 95% CIs from percentile bootstrap over per-prompt paired differences (2,000 resamples). Temperature 1.0, EOS masked.

	web	chat	code
99M	-0.0540 [-0.0582, -0.0501]	-0.0659 [-0.0681, -0.0635]	-0.0628 [-0.0654, -0.0601]
190M	-0.0513 [-0.0547, -0.0488]	-0.0522 [-0.0545, -0.0494]	-0.0497 [-0.0524, -0.0472]
390M	-0.0568 [-0.0598, -0.0545]	-0.0495 [-0.0511, -0.0478]	-0.0437 [-0.0460, -0.0415]
560M	-0.0481 [-0.0514, -0.0457]	-0.0416 [-0.0432, -0.0400]	-0.0386 [-0.0406, -0.0364]
830M	-0.0443 [-0.0473, -0.0420]	-0.0390 [-0.0406, -0.0374]	-0.0339 [-0.0357, -0.0322]
1.1B	-0.0378 [-0.0403, -0.0358]	-0.0352 [-0.0368, -0.0337]	-0.0291 [-0.0308, -0.0272]
1.5B	-0.0348 [-0.0370, -0.0330]	-0.0293 [-0.0306, -0.0281]	-0.0254 [-0.0270, -0.0237]

Table 5: α penalty of a ternary draft against the ternary target at matched parameter count, $\alpha(\text{TriLM}) - \alpha(\text{FloatLM})$, for all domains. 95% CIs from percentile bootstrap over per-prompt paired differences (2,000 resamples). Temperature 1.0, EOS masked.

	web	chat	code
99M	-0.0515 [-0.0550, -0.0485]	-0.0605 [-0.0627, -0.0582]	-0.0582 [-0.0607, -0.0558]
190M	-0.0446 [-0.0464, -0.0430]	-0.0470 [-0.0488, -0.0452]	-0.0429 [-0.0452, -0.0406]
390M	-0.0434 [-0.0447, -0.0422]	-0.0381 [-0.0397, -0.0366]	-0.0335 [-0.0354, -0.0316]
560M	-0.0302 [-0.0315, -0.0290]	-0.0266 [-0.0280, -0.0251]	-0.0254 [-0.0269, -0.0240]
830M	-0.0219 [-0.0230, -0.0208]	-0.0200 [-0.0212, -0.0188]	-0.0181 [-0.0194, -0.0167]
1.1B	-0.0112 [-0.0123, -0.0100]	-0.0127 [-0.0139, -0.0115]	-0.0115 [-0.0129, -0.0100]
1.5B	-0.0031 [-0.0045, -0.0016]	-0.0028 [-0.0039, -0.0017]	-0.0025 [-0.0040, -0.0009]

Table 6: Predicted speedup over target-only decoding for every draft, target, and domain, $k = 5$. Computed from mean α and the memory-bound cost model. Footprints assume 2-bit ternary projections and FP16 elsewhere.

target	family	MB	web	chat	code
FloatLM_3.9B	FloatLM	200	2.17	2.42	2.73
		381	2.16	2.39	2.66
		785	2.01	2.19	2.40
		1138	1.82	1.98	2.16
		1668	1.59	1.73	1.86
		2299	1.37	1.49	1.59
		3031	1.18	1.27	1.36
	PTQ	115	0.95	0.94	0.94
		183	0.93	0.92	0.91
		278	0.91	0.89	0.87
		368	0.85	0.84	0.83
		479	0.84	0.82	0.79
		604	0.76	0.75	0.74
		740	0.74	0.72	0.71
	TriLM	115	2.05	2.22	2.50
		183	2.16	2.37	2.65
		278	2.26	2.49	2.76
		368	2.28	2.50	2.75
		479	2.27	2.48	2.71
		604	2.23	2.43	2.64
		740	2.16	2.35	2.54
TriLM_3.9B	FloatLM	200	1.51	1.66	1.88
		381	1.21	1.32	1.48
		785	0.85	0.91	1.01
		1138	0.66	0.71	0.79
		1668	0.50	0.54	0.59
		2299	0.39	0.42	0.45
		3031	0.31	0.33	0.36
	PTQ	115	0.74	0.74	0.73
		183	0.65	0.65	0.64
		278	0.56	0.55	0.54
		368	0.48	0.47	0.46
		479	0.43	0.42	0.41
		604	0.35	0.35	0.35
		740	0.32	0.31	0.31
	TriLM	115	1.64	1.75	1.99
		183	1.55	1.68	1.89
		278	1.44	1.56	1.74
		368	1.32	1.43	1.58
		479	1.20	1.29	1.42
		604	1.08	1.15	1.27
		740	0.97	1.04	1.13

Table 7: Analytical α vs. empirical acceptance from end-to-end speculative decoding, with predicted & measured tokens per round and their gaps. 9 drafts, 3 domains, 256 prompts, FloatLM_3.9B target, $k = 5$.

draft	domain	analytical alpha	empirical alpha	pred. tok/rnd	meas. tok/rnd	alpha gap	tok/rnd gap
FloatLM_1.5B	chat	0.7983	0.7907	3.6982	3.6192	-0.0076	-0.0789
	code	0.8217	0.8201	3.9712	3.8955	-0.0016	-0.0758
	web	0.7725	0.7669	3.4948	3.4410	-0.0056	-0.0538
FloatLM_560M	chat	0.7611	0.7534	3.4009	3.3285	-0.0076	-0.0723
	code	0.7956	0.7889	3.7448	3.6122	-0.0067	-0.1326
	web	0.7332	0.7325	3.2047	3.1835	-0.0007	-0.0212
FloatLM_99M	chat	0.6626	0.6711	2.7545	2.7761	0.0085	0.0217
	code	0.7124	0.7050	3.1272	3.0081	-0.0074	-0.1191
	web	0.6215	0.6264	2.5371	2.5480	0.0049	0.0109
PTQ_1.5B	chat	0.0525	0.0508	1.0564	1.0544	-0.0018	-0.0020
	code	0.0354	0.0356	1.0375	1.0377	0.0002	0.0002
	web	0.0791	0.0813	1.0869	1.0896	0.0022	0.0028
PTQ_560M	chat	0.0274	0.0246	1.0285	1.0256	-0.0028	-0.0029
	code	0.0166	0.0165	1.0172	1.0171	-0.0001	-0.0001
	web	0.0399	0.0397	1.0420	1.0420	-0.0002	0.0000
PTQ_99M	chat	0.0105	0.0098	1.0107	1.0100	-0.0007	-0.0007
	code	0.0049	0.0038	1.0050	1.0038	-0.0011	-0.0011
	web	0.0181	0.0184	1.0186	1.0189	0.0003	0.0004
TriLM_1.5B	chat	0.7681	0.7645	3.4541	3.4156	-0.0036	-0.0385
	code	0.7978	0.7950	3.7627	3.6974	-0.0028	-0.0653
	web	0.7392	0.7374	3.2494	3.2297	-0.0018	-0.0198
TriLM_560M	chat	0.7207	0.7193	3.1124	3.0773	-0.0014	-0.0351
	code	0.7571	0.7439	3.4413	3.2464	-0.0132	-0.1949
	web	0.6872	0.6845	2.8997	2.8718	-0.0027	-0.0279
TriLM_99M	chat	0.5971	0.5942	2.4152	2.3739	-0.0029	-0.0413
	code	0.6502	0.6432	2.7455	2.6412	-0.0069	-0.1043
	web	0.5679	0.5666	2.2692	2.2565	-0.0014	-0.0127